Exercise 3 — Comparison: Fuzzing (Ex2) vs Symbolic Execution (Ex3)

1. Coverage on the four target functions

Both techniques covered the vast majority of the four target functions, with uncovered lines limited to those blocked by known bugs or genuinely unreachable code (tree_max, CC_ERR_ALLOC paths). AFL++ achieved higher overall coverage, naturally reaching rare tree shapes and internal helper sub-functions such as the parent-walk loop in get_successor_node that KLEE missed within its path budget. KLEE reached 87% line coverage and 92% function coverage, covering all main paths through the target functions and additionally exercising treetable_destroy and the RL zigzag rebalancing case.

2. Bugs detected

The techniques found complementary bugs. KLEE found two precise API-contract violations: treetable_get_greater_than returns CC_OK with garbage data instead of CC_ERR_KEY_NOT_FOUND when queried on the maximum key (BUG-001), and treetable_get_first_key crashes inside tree_min when called on an empty table (BUG-002). AFL++ found a different class: 22 crashing inputs revealing RB-tree invariant violations (balanced && sorted) under multi-step insertion patterns. These logic bugs were missed by KLEE because the symbolic tests used small, property-focused sequences that did not reach the tree shapes required to trigger the violation. The two techniques therefore found complementary bugs: KLEE produced precise, minimal counterexamples for edge-case API violations, while fuzzing exposed deep structural invariant failures under complex sequences.

3. Test-generation cost

AFL++ ran for 30 minutes in persistent mode with AddressSanitizer, producing 69 minimised tests after afl-cmin. KLEE completed all symbolic tests in under a minute, yielding 14 concrete test files. The fuzzing suite is larger and more diverse; the KLEE suite is smaller and more focused.

4. Effort to write the harness / symbolic tests

The AFL++ harness was straightforward: decide an input format, ensure all four target functions are reachable, and prepare seeds that exercise each at least once. The symbolic test suite required more upfront design: each test had to be structured around a specific property, with klee_assume constraints chosen carefully to guide KLEE toward the intended paths, while staying within its path budget. The property-based structure made the intent of each test explicit but required deeper knowledge of the implementation's invariants.

5. Conclusion

For this case study, fuzzing is the better first choice. It requires minimal setup, achieves higher line coverage, and automatically uncovered a class of deep RB-tree invariant violations that would be very difficult to target with hand-written symbolic tests. Symbolic execution with KLEE is a valuable complement for systematically verifying specific API contracts and edge cases, producing precise, minimal counterexamples with exact line-level diagnostics. The ideal workflow is to fuzz first to find structural bugs quickly, then apply KLEE to verify targeted properties.